

has been improved several times, and the fastest algorithm is now by Williams [457], with a running time of $O(V^3/2^{\Omega(\lg^{1/2} V)})$.

Another line of research demonstrates how to apply algorithms for fast matrix multiplication (see the chapter notes for Chapter 4) to the all-pairs shortest paths problem. Let $O(n^\omega)$ be the running time of the fastest algorithm for multiplying two $n \times n$ matrices. Galil and Margalit [170, 171] and Seidel [403] designed algorithms that solve the all-pairs shortest paths problem in undirected, unweighted graphs in $(V^\omega p(V))$ time, where $p(n)$ denotes a particular function that is polylogarithmically bounded in n . In dense graphs, these algorithms are faster than the $O(VE)$ time needed to perform $|V|$ breadth-first searches. Several researchers have extended these results to give algorithms for solving the all-pairs shortest paths problem in undirected graphs in which the edge weights are integers in the range $\{1, 2, \dots, W\}$. The asymptotically fastest such algorithm, by Shoshan and Zwick [410], runs in $O(WV^\omega p(VW))$ time. In directed graphs, the best algorithm to date is due to Zwick [467] and runs in $\tilde{O}(W^{1/(4-\omega)} V^{2+1/(4-\omega)})$ time.

Karger, Koller, and Phillips [244] and independently McGeoch [320] have given a time bound that depends on E^* , the set of edges in E that participate in some shortest path. Given a graph with nonnegative edge weights, their algorithms run in $O(VE^* + V^2 \lg V)$ time and improve upon running Dijkstra's algorithm $|V|$ times when $|E^*| = o(E)$. Pettie [355] uses an approach based on component hierarchies to achieve a running time of $O(VE + V^2 \lg \lg V)$, and the same running time is also achieved by Hagerup [205].

Baswana, Hariharan, and Sen [37] examined decremental algorithms, which allow a sequence of intermixed edge deletions and queries, for maintaining all-pairs shortest paths and transitive-closure information. When a path exists, their randomized transitive-closure algorithm can fail to report it with probability $1/n^c$ for an arbitrary $c > 0$. The query times are $O(1)$ with high probability. For transitive closure, the amortized time for each update is $O(V^{4/3} \lg^{1/3} V)$. By comparison, Problem 23-1, in which edges are inserted, asks for an incremental algorithm. For all-pairs shortest paths, the update times depend on the queries. For queries just giving the shortest-path weights, the amortized time per update is $O(V^3/E \lg^2 V)$. To report the actual shortest path, the amortized update time is $\min \{O(V^{3/2} \sqrt{\lg V}), O(V^3/E \lg^2 V)\}$. Demetrescu and Italiano [111] showed how to handle update and query operations when edges are both inserted and deleted, as long as the range of edge weights is bounded.

Aho, Hopcroft, and Ullman [5] defined an algebraic structure known as a “closed semiring,” which serves as a general framework for solving path problems in directed graphs. Both the Floyd-Warshall algorithm and the transitive-closure algorithm from Section 23.2 are instantiations of an all-pairs algorithm based on closed semirings. Maggs and Plotkin [309] showed how to find minimum spanning trees using a closed semiring.

Just as you can model a road map as a directed graph in order to find the shortest path from one point to another, you can also interpret a directed graph as a “flow network” and use it to answer questions about material flows. Imagine a material coursing through a system from a source, where the material is produced, to a sink, where it is consumed. The source produces the material at some steady rate, and the sink consumes the material at the same rate. The “flow” of the material at any point in the system is intuitively the rate at which the material moves. Flow networks can model many problems, including liquids flowing through pipes, parts through assembly lines, current through electrical networks, and information through communication networks.

You can think of each directed edge in a flow network as a conduit for the material. Each conduit has a stated capacity, given as a maximum rate at which the material can flow through the conduit, such as 200 gallons of liquid per hour through a pipe or 20 amperes of electrical current through a wire. Vertices are conduit junctions, and other than the source and sink, material flows through the vertices without collecting in them. In other words, the rate at which material enters a vertex must equal the rate at which it leaves the vertex. We call this property “flow conservation,” and it is equivalent to Kirchhoff’s current law when the material is electrical current.

The goal of the maximum-flow problem is to compute the greatest rate for shipping material from the source to the sink without violating any capacity constraints. It is one of the simplest problems concerning flow networks and, as we shall see in this chapter, this problem can be solved by efficient algorithms. Moreover, other network-flow problems are solvable by adapting the basic techniques used in maximum-flow algorithms.

This chapter presents two general methods for solving the maximum-flow problem. Section 24.1 formalizes the notions of flow networks and flows, formally defining the maximum-flow problem. Section 24.2 describes the classical method

of Ford and Fulkerson for finding maximum flows. We finish up with a simple application of this method, finding a maximum matching in an undirected bipartite graph, in Section 24.3. (Section 25.1 will give a more efficient algorithm that is specifically designed to find a maximum matching in a bipartite graph.)

24.1 Flow networks

This section gives a graph-theoretic definition of flow networks, discusses their properties, and defines the maximum-flow problem precisely. It also introduces some helpful notation.

Flow networks and flows

A **flow network** $G = (V, E)$ is a directed graph in which each edge $(u, v) \in E$ has a nonnegative **capacity** $c(u, v) \geq 0$. We further require that if E contains an edge (u, v) , then there is no edge (v, u) in the reverse direction. (We'll see shortly how to work around this restriction.) If $(u, v) \notin E$, then for convenience we define $c(u, v) = 0$, and we disallow self-loops. Each flow network contains two distinguished vertices: a **source** s and a **sink** t . For convenience, we assume that each vertex lies on some path from the source to the sink. That is, for each vertex $v \in V$, the flow network contains a path $s \rightsquigarrow v \rightsquigarrow t$. Because each vertex other than s has at least one entering edge, we have $|E| \geq |V| - 1$. Figure 24.1 shows an example of a flow network.

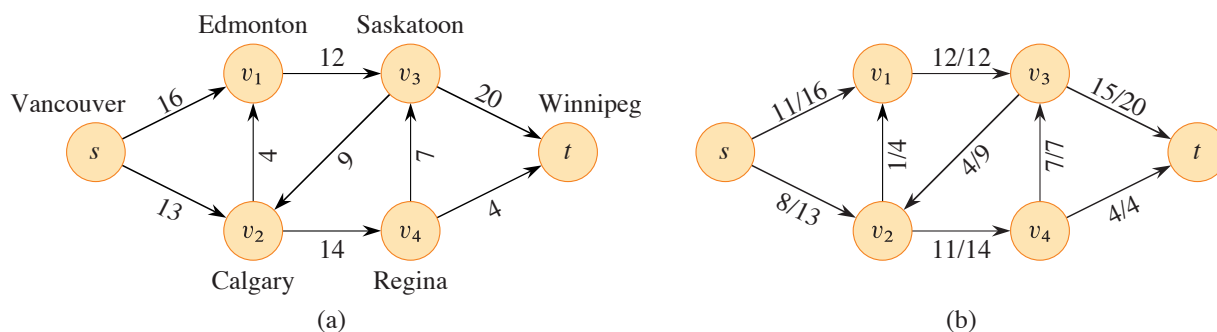


Figure 24.1 (a) A flow network $G = (V, E)$ for the Lucky Puck Company's trucking problem. The Vancouver factory is the source s , and the Winnipeg warehouse is the sink t . The company ships pucks through intermediate cities, but only $c(u, v)$ crates per day can go from city u to city v . Each edge is labeled with its capacity. (b) A flow f in G with value $|f| = 19$. Each edge (u, v) is labeled by $f(u, v)/c(u, v)$. The slash notation merely separates the flow and capacity and does not indicate division.

We are now ready to define flows more formally. Let $G = (V, E)$ be a flow network with a capacity function c . Let s be the source of the network, and let t be the sink. A **flow** in G is a real-valued function $f : V \times V \rightarrow \mathbb{R}$ that satisfies the following two properties:

Capacity constraint: For all $u, v \in V$, we require

$$0 \leq f(u, v) \leq c(u, v) .$$

The flow from one vertex to another must be nonnegative and must not exceed the given capacity.

Flow conservation: For all $u \in V - \{s, t\}$, we require

$$\sum_{v \in V} f(v, u) = \sum_{v \in V} f(u, v) .$$

The total flow into a vertex other than the source or sink must equal the total flow out of that vertex—informally, “flow in equals flow out.”

When $(u, v) \notin E$, there can be no flow from u to v , and $f(u, v) = 0$.

We call the nonnegative quantity $f(u, v)$ the flow from vertex u to vertex v . The **value** $|f|$ of a flow f is defined as

$$|f| = \sum_{v \in V} f(s, v) - \sum_{v \in V} f(v, s) , \quad (24.1)$$

that is, the total flow out of the source minus the flow into the source. (Here, the $|\cdot|$ notation denotes flow value, not absolute value or cardinality.) Typically, a flow network does not have any edges into the source, and the flow into the source, given by the summation $\sum_{v \in V} f(v, s)$, is 0. We include it, however, because when we introduce residual networks later in this chapter, the flow into the source can be positive. In the **maximum-flow problem**, the input is a flow network G with source s and sink t , and the goal is to find a flow of maximum value.

An example of flow

A flow network can model the trucking problem shown in Figure 24.1(a). The Lucky Puck Company has a factory (source s) in Vancouver that manufactures hockey pucks, and it has a warehouse (sink t) in Winnipeg that stocks them. Lucky Puck leases space on trucks from another firm to ship the pucks from the factory to the warehouse. Because the trucks travel over specified routes (edges) between cities (vertices) and have a limited capacity, Lucky Puck can ship at most $c(u, v)$ crates per day between each pair of cities u and v in Figure 24.1(a). Lucky Puck

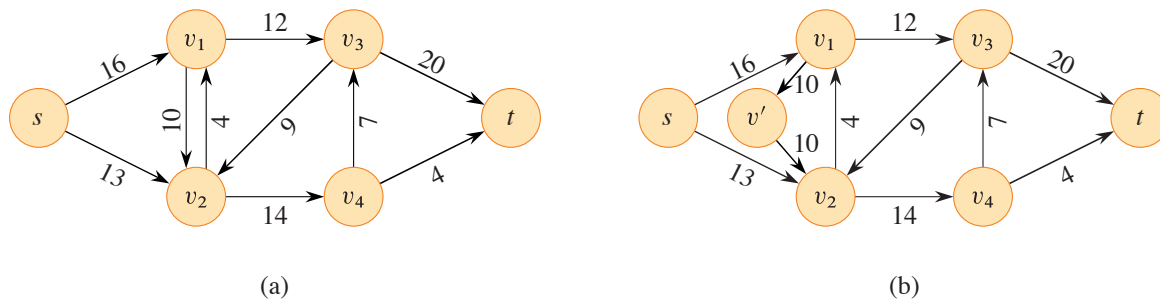


Figure 24.2 Converting a network with antiparallel edges to an equivalent one with no antiparallel edges. **(a)** A flow network containing both the edges (v_1, v_2) and (v_2, v_1) . **(b)** An equivalent network with no antiparallel edges. A new vertex v' was added, and edge (v_1, v_2) was replaced by the pair of edges (v_1, v') and (v', v_2) , both with the same capacity as (v_1, v_2) .

has no control over these routes and capacities, and so the company cannot alter the flow network shown in Figure 24.1(a). They need to determine the largest number p of crates per day that they can ship and then to produce this amount, since there is no point in producing more pucks than they can ship to their warehouse. Lucky Puck is not concerned with how long it takes for a given puck to get from the factory to the warehouse. They care only that p crates per day leave the factory and p crates per day arrive at the warehouse.

A flow in this network models the “flow” of shipments because the number of crates shipped per day from one city to another is subject to a capacity constraint. Additionally, the model must obey flow conservation, for in a steady state, the rate at which pucks enter an intermediate city must equal the rate at which they leave. Otherwise, crates would accumulate at intermediate cities.

Modeling problems with antiparallel edges

Suppose that the trucking firm offers Lucky Puck the opportunity to lease space for 10 crates in trucks going from Edmonton to Calgary. It might seem natural to add this opportunity to our example and form the network shown in Figure 24.2(a). This network suffers from one problem, however: it violates the original assumption that if edge $(v_1, v_2) \in E$, then $(v_2, v_1) \notin E$. We call the two edges (v_1, v_2) and (v_2, v_1) *antiparallel*. Thus, to model a flow problem with antiparallel edges, the network must be transformed into an equivalent one containing no antiparallel edges. Figure 24.2(b) displays this equivalent network. To transform the network, choose one of the two antiparallel edges, in this case (v_1, v_2) , and split it by adding a new vertex v' and replacing edge (v_1, v_2) with the pair of edges (v_1, v') and (v', v_2) . Also set the capacity of both new edges to the capacity of the original edge. The resulting network satisfies the property that if an edge belongs to

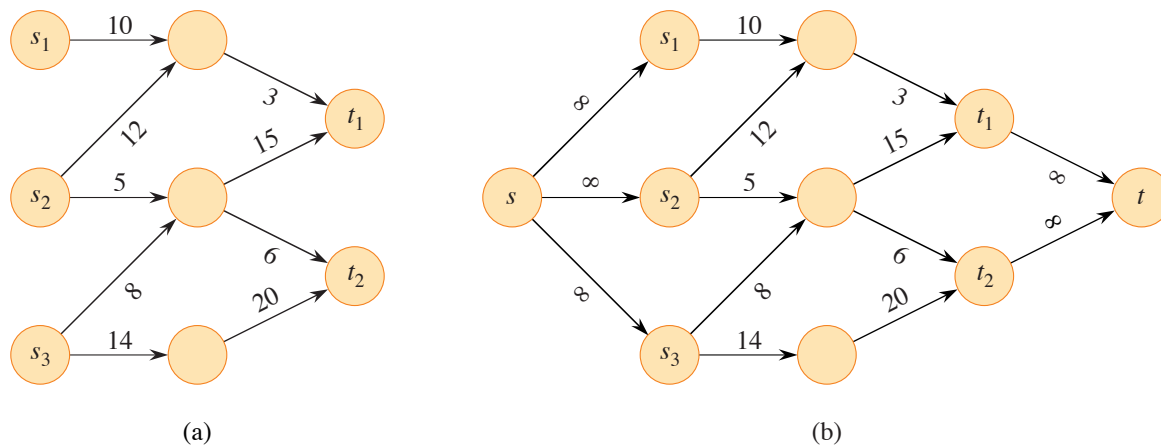


Figure 24.3 Converting a multiple-source, multiple-sink maximum-flow problem into a problem with a single source and a single sink. (a) A flow network with three sources $S = \{s_1, s_2, s_3\}$ and two sinks $T = \{t_1, t_2\}$. (b) An equivalent single-source, single-sink flow network. Add a supersource s and an edge with infinite capacity from s to each of the multiple sources. Also add a supersink t and an edge with infinite capacity from each of the multiple sinks to t .

the network, the reverse edge does not. As Exercise 24.1-1 asks you to prove, the resulting network is equivalent to the original one.

Networks with multiple sources and sinks

A maximum-flow problem may have several sources and sinks, rather than just one of each. The Lucky Puck Company, for example, might actually have a set of m factories $\{s_1, s_2, \dots, s_m\}$ and a set of n warehouses $\{t_1, t_2, \dots, t_n\}$, as shown in Figure 24.3(a). Fortunately, this problem is no harder than ordinary maximum flow.

The problem of determining a maximum flow in a network with multiple sources and multiple sinks reduces to an ordinary maximum-flow problem. Figure 24.3(b) shows how to convert the network from (a) to an ordinary flow network with only a single source and a single sink. Add a **supersource** s and add a directed edge (s, s_i) with capacity $c(s, s_i) = \infty$ for each $i = 1, 2, \dots, m$. Similarly, create a new **supersink** t and add a directed edge (t_i, t) with capacity $c(t_i, t) = \infty$ for each $i = 1, 2, \dots, n$. Intuitively, any flow in the network in (a) corresponds to a flow in the network in (b), and vice versa. The single supersource s provides as much flow as desired for the multiple sources s_i , and the single supersink t likewise consumes as much flow as desired for the multiple sinks t_i . Exercise 24.1-2 asks you to prove formally that the two problems are equivalent.

Exercises

24.1-1

Show that splitting an edge in a flow network yields an equivalent network. More formally, suppose that flow network G contains edge (u, v) , and define a new flow network G' by creating a new vertex x and replacing (u, v) by new edges (u, x) and (x, v) with $c(u, x) = c(x, v) = c(u, v)$. Show that a maximum flow in G' has the same value as a maximum flow in G .

24.1-2

Extend the flow properties and definitions to the multiple-source, multiple-sink problem. Show that any flow in a multiple-source, multiple-sink flow network corresponds to a flow of identical value in the single-source, single-sink network obtained by adding a supersource and a supersink, and vice versa.

24.1-3

Suppose that a flow network $G = (V, E)$ violates the assumption that the network contains a path $s \rightsquigarrow v \rightsquigarrow t$ for all vertices $v \in V$. Let u be a vertex for which there is no path $s \rightsquigarrow u \rightsquigarrow t$. Show that there must exist a maximum flow f in G such that $f(u, v) = f(v, u) = 0$ for all vertices $v \in V$.

24.1-4

Let f be a flow in a network, and let α be a real number. The *scalar flow product*, denoted αf , is a function from $V \times V$ to $\mathbb{R}$ defined by

$$(\alpha f)(u, v) = \alpha \cdot f(u, v) .$$

Prove that the flows in a network form a *convex set*. That is, show that if f_1 and f_2 are flows, then so is $\alpha f_1 + (1 - \alpha)f_2$ for all α in the range $0 \leq \alpha \leq 1$.

24.1-5

State the maximum-flow problem as a linear-programming problem.

24.1-6

Professor Adam has two children who, unfortunately, dislike each other. The problem is so severe that not only do they refuse to walk to school together, but in fact each one refuses to walk on any block that the other child has stepped on that day. The children have no problem with their paths crossing at a corner. Fortunately both the professor's house and the school are on corners, but beyond that he is not sure if it is going to be possible to send both of his children to the same school. The professor has a map of his town. Show how to formulate the problem of determining whether both his children can go to the same school as a maximum-flow problem.

24.1-7

Suppose that, in addition to edge capacities, a flow network has *vertex capacities*. That is each vertex v has a limit $l(v)$ on how much flow can pass through v . Show how to transform a flow network $G = (V, E)$ with vertex capacities into an equivalent flow network $G' = (V', E')$ without vertex capacities, such that a maximum flow in G' has the same value as a maximum flow in G . How many vertices and edges does G' have?

24.2 The Ford-Fulkerson method

This section presents the Ford-Fulkerson method for solving the maximum-flow problem. We call it a “method” rather than an “algorithm” because it encompasses several implementations with differing running times. The Ford-Fulkerson method depends on three important ideas that transcend the method and are relevant to many flow algorithms and problems: residual networks, augmenting paths, and cuts. These ideas are essential to the important max-flow min-cut theorem (Theorem 24.6), which characterizes the value of a maximum flow in terms of cuts of the flow network. We end this section by presenting one specific implementation of the Ford-Fulkerson method and analyzing its running time.

The Ford-Fulkerson method iteratively increases the value of the flow. It starts with $f(u, v) = 0$ for all $u, v \in V$, giving an initial flow of value 0. Each iteration increases the flow value in G by finding an “augmenting path” in an associated “residual network” G_f . The edges of the augmenting path in G_f indicate on which edges in G to update the flow in order to increase the flow value. Although each iteration of the Ford-Fulkerson method increases the value of the flow, we’ll see that the flow on any particular edge of G may increase or decrease. Although it might seem counterintuitive to decrease the flow on an edge, doing so may enable flow to increase on other edges, allowing more flow to travel from the source to the sink. The Ford-Fulkerson method, given in the procedure FORD-FULKERSON-METHOD, repeatedly augments the flow until the residual network has no more augmenting paths. The max-flow min-cut theorem shows that upon termination, this process yields a maximum flow.

FORD-FULKERSON-METHOD(G, s, t)

```

1  initialize flow  $f$  to 0
2  while there exists an augmenting path  $p$  in the residual network  $G_f$ 
3      augment flow  $f$  along  $p$ 
4  return  $f$ 
```

In order to implement and analyze the Ford-Fulkerson method, we need to introduce several additional concepts.

Residual networks

Intuitively, given a flow network G and a flow f , the residual network G_f consists of edges whose capacities represent how the flow can change on edges of G . An edge of the flow network can admit an amount of additional flow equal to the edge's capacity minus the flow on that edge. If that value is positive, that edge goes into G_f with a “residual capacity” of $c_f(u, v) = c(u, v) - f(u, v)$. The only edges of G that belong to G_f are those that can admit more flow. Those edges (u, v) whose flow equals their capacity have $c_f(u, v) = 0$, and they do not belong to G_f .

You might be surprised that the residual network G_f can also contain edges that are not in G . As an algorithm manipulates the flow, with the goal of increasing the total flow, it might need to decrease the flow on a particular edge in order to increase the flow elsewhere. In order to represent a possible decrease in the positive flow $f(u, v)$ on an edge in G , the residual network G_f contains an edge (v, u) with residual capacity $c_f(v, u) = f(u, v)$ —that is, an edge that can admit flow in the opposite direction to (u, v) , at most canceling out the flow on (u, v) . These reverse edges in the residual network allow an algorithm to send back flow it has already sent along an edge. Sending flow back along an edge is equivalent to *decreasing* the flow on the edge, which is a necessary operation in many algorithms.

More formally, for a flow network $G = (V, E)$ with source s , sink t , and a flow f , consider a pair of vertices $u, v \in V$. We define the **residual capacity** $c_f(u, v)$ by

$$c_f(u, v) = \begin{cases} c(u, v) - f(u, v) & \text{if } (u, v) \in E, \\ f(v, u) & \text{if } (v, u) \in E, \\ 0 & \text{otherwise.} \end{cases} \quad (24.2)$$

In a flow network, $(u, v) \in E$ implies $(v, u) \notin E$, and so exactly one case in equation (24.2) applies to each ordered pair of vertices.

As an example of equation (24.2), if $c(u, v) = 16$ and $f(u, v) = 11$, then $f(u, v)$ can increase by up to $c_f(u, v) = 5$ units before exceeding the capacity constraint on edge (u, v) . Alternatively, up to 11 units of flow can return from v to u , so that $c_f(v, u) = 11$.

Given a flow network $G = (V, E)$ and a flow f , the **residual network** of G induced by f is $G_f = (V, E_f)$, where

$$E_f = \{(u, v) \in V \times V : c_f(u, v) > 0\}. \quad (24.3)$$

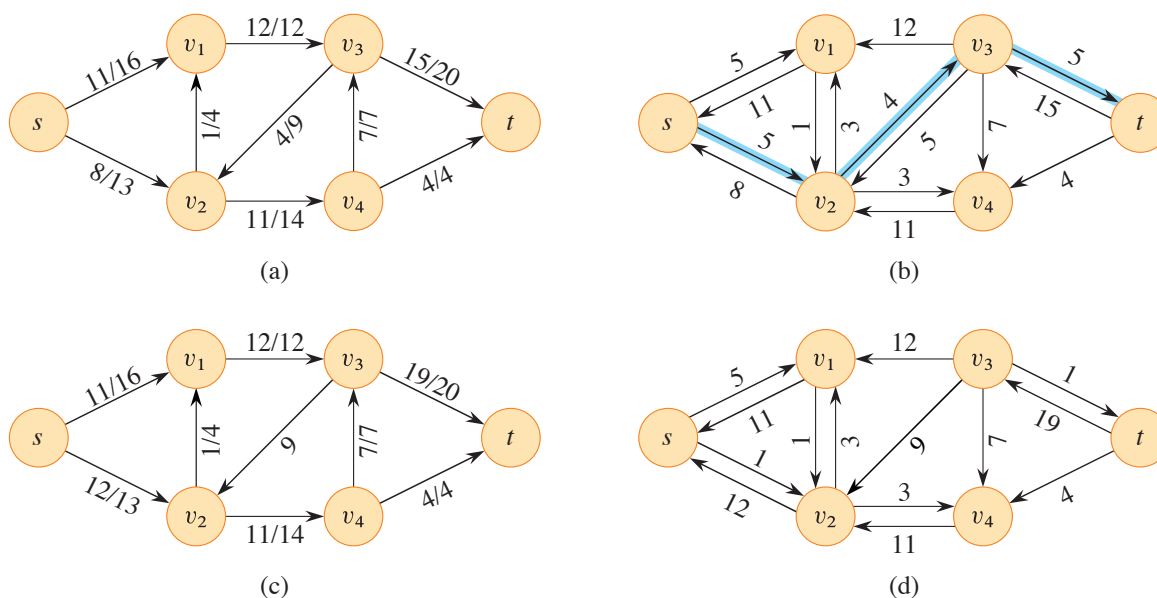


Figure 24.4 (a) The flow network G and flow f of Figure 24.1(b). (b) The residual network G_f with augmenting path p , having residual capacity $c_f(p) = c_f(v_2, v_3) = 4$, in blue. Edges with residual capacity equal to 0, such as (v_1, v_3) , are not shown, a convention we follow in the remainder of this section. (c) The flow in G that results from augmenting along path p by its residual capacity 4. Edges carrying no flow, such as (v_3, v_2) , are labeled only by their capacity, another convention we follow throughout. (d) The residual network induced by the flow in (c).

That is, as promised above, each edge of the residual network, or *residual edge*, can admit a flow that is greater than 0. Figure 24.4(a) repeats the flow network G and flow f of Figure 24.1(b), and Figure 24.4(b) shows the corresponding residual network G_f . The edges in E_f are either edges in E or their reversals, and thus

$$|E_f| \leq 2 |E| .$$

Observe that the residual network G_f is similar to a flow network with capacities given by c_f . It does not satisfy the definition of a flow network, however, because it could contain antiparallel edges. Other than this difference, a residual network has the same properties as a flow network, and we can define a flow in the residual network as one that satisfies the definition of a flow, but with respect to capacities c_f in the residual network G_f .

A flow in a residual network provides a roadmap for adding flow to the original flow network. If f is a flow in G and f' is a flow in the corresponding residual network G_f , we define $f \uparrow f'$, the *augmentation* of flow f by f' , to be a function from $V \times V$ to $\mathbb{R}$, defined by